Phloem: Recursive Rewards for AI Architecture Research

An application-layer protocol that settles SAP on the Deterministic State Machine

@pepedzekichen

hello@pinealab.com

November 2026

Licensed under CC BY 4.0

Status. Open design draft for comment. This is a protocol specification, and no implementation exists yet. This version specifies object identity, the multi-parent reward split and its emissions bound, home-vault settlement with spent sets, and the steward model. Open problems, including a bound on Sybil gain, fertility weight calibration, the similarity function, and allocation of the SAP supply, are listed in Section 10. Contributors and reviewers are welcome: contact hello@pinealab.com or @pepedzekichen on X, Telegram, and Discord.

Abstract

Architectural primitives such as attention variants, routing functions, and memory structures are reused across many AI systems. The credit they receive is mostly fixed at release, through benchmark scores and early citations, and says little about what later work they made possible. Phloem registers such primitives as content-addressed objects and pays them when registered downstream work builds on them. Objects live in Phloem vaults, provenance registries each rooted at one object and kept by a steward. Peers score each object's fertility inside its vault from the structure of its descendants and its continued use. Each activation's reward moves upstream along declared ancestry and is split among parents at every hop, so one activation emits less than $R_{\text{activation}}/(1-\rho)$ on any provenance DAG. Each activation is settled once, by the vault that records it, against a spent set any peer can audit. Credit is paid in SAP, a token on the Deterministic State Machine (DSM) [1], which prevents double spending without a global ledger; provenance, scoring, and settlement are Phloem application logic checked by peers. Measuring fertility well enough to resist gaming is the main open problem.

1. Introduction

Much of current AI is assembled from reusable architectural primitives: transformer attention [2], mixture-of-experts routers, gradient estimators for discrete operations, memory-augmented controllers. Later systems fork them, modify them, and combine them. Credit for that reuse has three gaps.

- Search is expensive. Labs that can afford architecture search [3, 4] release the winner and keep the trajectory. Phloem lets the intermediate architectures of a search run be registered as objects, so publishing them can earn credit when others build on them.

- Credit is read early. Leaderboard rank, conference acceptance, and citation counts are mostly settled within a year or two of release, before anyone knows which ideas later work will depend on.
- Credit does not flow backward. Citation graphs and package dependency graphs record part of a lineage, but nothing pays along it to the work that later results depended on.

Phloem uses fertility, an object's capacity to enable later work, as its long-term credit signal. The protocol (i) defines a content-addressed format for Phloem Objects, (ii) records fork, mutation, composition, and supersession relationships in Phloem vaults, (iii) computes fertility inside each vault, (iv) pays SAP credit upstream along provenance with a fixed per-hop split, and (v) relies for integrity on content hashes, signatures under DSM device keys, per-vault spent sets, and DSM acceptance of every SAP transfer.

Phloem is an application-layer provenance and attribution protocol that settles SAP on DSM [1]. SAP balances, SAP transfers, and escrowed collateral are DSM state. Objects, edges, activation records, spent sets, fertility scores, and settlement records are Phloem data, kept by each vault's steward as signed records that any peer can verify (Section 3.2). Ordinary operation needs no globally replicated ledger.

1.1 Illustrative walkthrough

The numbers here are illustrative. Peer A registers Root, a sparse attention pattern with no parents. Its identifier is the BLAKE3 hash of its canonical serialization, and vault $V(\text{Root})$ opens with Root as its root object. Later, peers register Child1, a mutation of Root that adds grouped-query attention, and Child2, a composition of Root with a routing primitive Q that lives in another vault. Each registrant pays the propagation fee c_{prop} in SAP to the Phloem treasury. Root has the smaller of Child2's two parent ids in this example, so Child2's home vault is $V(\text{Root})$.

Inside $V(\text{Root})$, suppose fertility $F(\text{Root}) = 0.55$, $F(\text{Child1}) = 0.30$, $F(\text{Child2}) = 0.15$, and an activation budget $R_{\text{activation}}$. Each object's base reward is its share of fertility. With upstream ratio $\rho = 0.25$, Child1 passes a quarter of its base reward to Root. Child2 passes the same fraction upstream, split equally between Root and Q. Root earns from later reuse even if it never led a benchmark. Section 2.6 works the numbers.

1.2 Design thesis

An architectural idea can score modestly on the benchmarks of its day and still enable much of the work that follows. Phloem bets that descendant structure is harder to fake than benchmark numbers. Section 5 shows where that bet is weakest: capability labels and Sybil descendants.

2. Mechanism: actions and reward flow

Five operations are deterministic functions of a vault's published history: RegisterObject, ForkObject, SupersedeObject, ComputeFertility, and DistributeRewards. Notation: o is an

object, $p(o)$ its parent set, $sup(o)$ the earlier version it supersedes, $A(o)$ its ancestors, $D(o)$ its descendants within depth τ, V a vault, $|V|$ the number of objects whose home is V, and home(o) the vault that pays o's base reward. Symbols are collected in Appendix A.0.

2.1 RegisterObject

Inputs are a machine-readable specification M, the creator's DSM identity H, a declared capability set Cap, a parent id set p (empty for a root), an optional superseded id sup, and an advisory timestamp t_{adv} that is not hashed. The identifier is a content hash:

$$id = \text{BLAKE3-256}\Big(\texttt{"phloem/object/v1\0"} \parallel \text{CanonicalSerialize}(M, H, Cap, p, sup)\Big)$$

Canonical serialization sorts p and covers every schema field, so two objects share an id only if their bytes are identical, except with negligible probability. The position at which the steward accepts an object is recorded in its registration record and is outside the hashed content.

Capabilities in Cap must be names defined in the vault's parameter set, each with a verification predicate ver_c fixed there (Section 6). A declared name outside that list scores nothing. A declared capability counts as verified at o when its predicate, re-executed on the committed artifact, returns 1; a peer that does not re-execute it relies on the creator's collateral (Section 6, σ_3).

The home vault is a pure function of content. For a root, $\text{home}(o) = V(o)$. Otherwise $\text{home}(o) = \text{home}(\min p(o))$, where min takes the lexicographically smallest parent id. A registration is valid only when the steward of home(o) records it (Section 3.2); other vaults may import the object as a reference.

A registration is rejected if its id is already present in the home vault, its bytes do not hash to its id, a parent has no valid registration record, it would create a cycle, M fails the schema, it is recorded outside home(o), or the registrant cannot pay c_{prop}. On accept, the registrant pays c_{prop} to the Phloem treasury account as a DSM transfer of SAP, and the steward appends o to the vault and publishes a signed registration record. The record carries the object bytes or their hash, parent inclusion proofs, a reference to the fee transfer, and the creator's signature under a DSM device key whose identity is proven by its DSM directory entry. Receiving peers check the record; DSM checks only the SAP transfer. Fees go to the treasury, not to the steward, so a steward gains nothing from fees paid in its own vault (Section 5, A7).

2.2 ForkObject and SupersedeObject

ForkObject is RegisterObject with at least one parent. Each provenance edge has a type: fork (parent specification kept, parameters changed), mutation (structural change), or composition (several parents combined). Provenance edges of all three types carry reward; supersedes edges carry none.

Objects are immutable. Any edit, including a corrected description or a new reference commit, is a new object with its own hash and a supersedes edge to the version it replaces: $sup(o') = id(o)$. SupersedeObject is valid only when o' and o have the same creator H; an edit by anyone else is

a fork. Supersedes edges do not count as descendant links for fertility, so republishing one's own object earns nothing. Existing descendants keep pointing at the version they built on.

2.3 ComputeFertility

Fertility measures what an object made possible later, inside its vault. Each component is normalized to $[0, 1]$ by dividing by its maximum over the objects whose home is the vault (0 if that maximum is 0), and the score is a weighted sum

$$F(o) = w_1 B_{\text{eff}}(o) + w_2 C(o) + w_3 Div(o) + w_4 P(o) + w_5 Val(o) + w_6 S(o) + w_7 Dep(o)$$
$$w_i \geq 0, \qquad \sum_i w_i = 1$$

Default component weights:

Component	Symbol	Weight	Measures
Branching	B_{eff}	0.25	Depth-decayed active descendants, discounted for near-copies, damped past N_{thresh}
New capabilities	C	0.15	Capabilities first verified at a direct child, credited to that child's parents
Derivative diversity	Div	0.10	Entropy of descendant object types
Persistence	P	0.15	Decayed count of descendant registrations per activation cycle
Downstream value	Val	0	Not defined in this version
Search-space expansion	S	0	Not defined in this version
Dependency centrality	Dep	0.35	Descendants with a new capability for which o is a structural dependency

Capability credit follows one rule. Dep is the main attribution term: it credits any ancestor, at any depth, that lies on every provenance path from a descendant with a new capability to a root, so credit reaches an ancestor only when that capability structurally required it. C is a smaller bonus to the direct parents of the descendant where the capability was first verified, divided among them with the split weights of Section 2.4. Deeper ancestors receive capability credit only through Dep. Section 4 and Appendix A.2 give the formulas.

Val and S have no operational definition yet and carry weight 0 until they get one (Section 10). A usable Val would need a measure of downstream value that wash trading cannot inflate.

There is no oracle of true capability: a predicate checks what the parameter set chose to check, and Section 5 states what this means for spoofing. Different vaults may compute different $F(o)$ for the same object, because each sees a different set of descendants. Home-vault settlement (Section 2.5) means only one of those values pays.

2.4 DistributeRewards

An activation cycle in a vault closes after a fixed number of accepted registration records (100 by default), and the steward then records a signed activation record. A vault therefore pays only as it grows; a quiet vault pays nothing until its current cycle fills. The vault whose steward

records an activation is its home vault and the only vault that pays it. Base reward is normalized over the objects whose home is that vault:

$$R_{\text{base}}(o) = R_{\text{activation}} \cdot \frac{F(o)}{\sum_{o' \colon \text{home}(o')=V} F(o')}$$

Upstream reward then moves hop by hop. Let $m_k(x)$ be the reward mass arriving at object x after k hops, and let split weights $s(c,a) \geq 0$ satisfy $\sum_{a \in p(c)} s(c,a) = 1$ for every object c that has parents:

$$\begin{aligned} m_0(x) &= R_{\text{base}}(x) && \text{for home}(x) = V \\ m_k(a) &= \rho \sum_{c \colon a \in p(c)} s(c,a)\, m_{k-1}(c) && \text{for } k = 1, \ldots, \tau \\ R_{\text{up}}(a) &= \textstyle\sum_{k=1}^{\tau} m_k(a) \\ R_{\text{total}}(a) &= R_{\text{base}}(a) + R_{\text{up}}(a) \end{aligned}$$

The default split is equal, $s(c,a) = 1/|p(c)|$. A vault's parameter set may select a fertility-weighted split, $s(c,a) = F(a)/\sum_{a' \in p(c)} F(a')$, using F values committed at the close of the previous activation cycle, so the split never depends on the scores it helps compute. Fertility is normalized per vault, so scores from different vaults are not comparable. The split is therefore equal whenever a child's parents do not all share one home vault, a parent's F is unavailable, or the parent sum is zero. An ancestor reached by several paths receives the mass of each path. Nothing is duplicated, because each child's outgoing mass is divided among its parents.

Emissions bound. Write $\mu_k = \sum_x m_k(x)$ for the total mass at hop k. Summing the recursion over a and using $\sum_{a \in p(c)} s(c,a) = 1$,

$$\mu_k = \rho \sum_{c \colon p(c) \neq \emptyset} m_{k-1}(c) \leq \rho\, \mu_{k-1}$$

Mass reaching a root has no parent to move to, which only lowers μ_k. Since $\mu_0 = R_{\text{activation}}$, $\mu_k \leq \rho^k R_{\text{activation}}$, and the total $\mathcal{E}$ emitted by one activation satisfies

$$\begin{aligned} \mathcal{E} = \sum_{k=0}^{\tau} \mu_k &\leq R_{\text{activation}} \cdot \frac{1 - \rho^{\tau+1}}{1 - \rho} \\ &< \frac{R_{\text{activation}}}{1 - \rho} \end{aligned}$$

for any provenance DAG, any parent counts, and either split rule. At the defaults $\rho = 0.25$ and $\tau = 5$, $\mathcal{E} \leq (1365/1024)\, R_{\text{activation}} \approx 1.33301\, R_{\text{activation}}$, below $(4/3)\, R_{\text{activation}}$. The argument needs only that split weights sum to 1 at every hop.

Ancestors gain from their descendants twice: through the descendant terms of their own F, which set their base share, and through the cascade, which passes on a fraction ρ of each descendant's realized base reward. The two measure different things. F scores the shape of an object's descendant tree, including descendants that earn little, while the cascade pays in proportion to what descendants actually earn. Both draw on the same $R_{\text{activation}}$, so the emissions bound is

unaffected; how large ρ should be relative to the descendant weights in F is a calibration question (Section 10).

2.5 Settlement, home vaults, and spent sets

Each activation settles exactly once, in its home vault, through one DistributeRewards settlement that pays every $R_{\text{total}}(a)$ under vesting (Section 6), including upstream mass to ancestors whose home is another vault. Other vaults may read the activation, but a settlement of it by any other vault is invalid.

Each settlement consumes a one-time key derived from the activation record:

$$K(act) = \text{BLAKE3-256}\big(\texttt{"phloem/spent/v1\textbackslash 0"} \parallel \text{record_id}(act)\big)$$

The steward keeps the vault's spent set as a sparse Merkle tree and publishes its root with every settlement record. A settlement is valid only if (i) home(act) is the vault being settled, (ii) the activation record is included in that vault's published history, (iii) a non-membership proof shows $K(act)$ absent from the spent set before the settlement, (iv) the credited amounts equal the deterministic recomputation of Section 2.4, and (v) the spent-set root published with the settlement includes $K(act)$. A second settlement of the same activation fails (iii). This is the absent-before, present-after pattern DSM uses for its own one-time resources [1], applied at the application layer: DSM does not evaluate (i) to (v), and every peer that receives a settlement record does. Each payout is an ordinary DSM transfer of SAP from the steward, so DSM guarantees that no SAP unit is spent twice and that total supply is conserved. A steward that signs two conflicting settlement records, or one that fails (i) to (v), produces signed evidence against itself (Section 6). Because base rewards are normalized only over objects whose home is the paying vault, no object draws base reward from two denominators.

2.6 Worked toy numbers

Take $R_{\text{activation}} = 1200$ SAP, the fertility values of Section 1.1, $\rho = 0.25$, $\tau = 5$, and the equal split. Then $R_{\text{base}}(\text{Root}) = 660$, $R_{\text{base}}(\text{Child1}) = 360$, $R_{\text{base}}(\text{Child2}) = 180$. At hop 1, Child1 passes $0.25 \times 360 = 90$ to Root. Child2 passes $0.25 \times 180 = 45$, split 22.5 to Root and 22.5 to Q. Root and Q are roots, so the cascade stops at hop 1. Root's credit from this activation before vesting is $660 + 90 + 22.5 = 772.5$ SAP. Q receives 22.5 SAP from $V(\text{Root})$, which pays this activation; Q's base reward comes only from activations of its own vault. Total emission is $1200 + 135 = 1335$ SAP, within the bound $1200 \times 1365/1024 \approx 1599.6$ SAP.

3. Architecture: Phloem on DSM

3.1 DSM foundation

DSM [1] is a value layer without a global ledger. Each state transition is local to a relationship between devices and is accepted by deterministic predicates over canonical bytes, proofs, and signatures, with no wall-clock time or global height in the predicate. Signatures use SPHINCS+ instantiated with BLAKE3, a hash-based scheme related to NIST's SLH-DSA [5] but not one of

its standardized parameter sets. DSM is in beta, and its specification has not been reviewed outside its authors. The DSM statements in this paper reflect the specification as of November 2026 [6, 7].

Everything in this section except SAP balances and transfers is Phloem data; DSM sees only SAP. Ordering inside a vault is position in the steward's signed history, where each record names the hash of the record before it. We write $(V, \ell, prev)$ for vault, position, and predecessor hash. Acceptance, cycle boundaries, vesting, and the emissions schedule are all expressed in this ordering, and wall-clock time never affects acceptance.

DSM's Tripwire theorem states that no realized history contains two accepted successors that consume the same resource [1]. Phloem inherits it for SAP: a steward cannot spend the same SAP twice. DSM does not order Phloem's records or exclude conflicting versions of them, so their single history rests on the steward rule of Section 3.2 and on peers checking the steward's signatures. The economic security of fertility scoring and reward distribution is Phloem's own obligation. No security audit of Phloem has been done.

Validity depends only on proofs a peer holds; peers need not agree on a global state.

3.2 Phloem vaults and stewards

A Phloem vault $V(o)$ is a provenance registry for the family of objects rooted at o: its objects, edges, fertility scores, settlement records, and spent set. It is unrelated to DSM's Deterministic Limbo Vaults, which hold one owner's value under precommitted release terms. A vault begins when a peer registers a root, and its identity is the root's id.

Each vault has one steward, a DSM account named in the root's registration, by default the registrant's. The steward is the vault's single writer: it records registrations and activation records in order, keeps the spent set, holds the SAP the vault pays out, and pays each settlement as ordinary DSM transfers. This follows DSM's guidance for applications: only scarce value is DSM state, and the application's own records stay application data [6, 8]. The steward signs every record under its DSM device key and publishes it, so any peer can recompute fertility and settlements from the published history. A steward posts integrity collateral before its vault receives treasury funds (Section 6). It can still refuse or delay records, and DSM accepts any SAP transfer the steward signs, so a payout that matches no valid settlement record is detected and attributed to the steward rather than prevented. Steward selection, exit, and replacement are open (Section 10).

A steward can also stop working. If it refuses a registration, the registrant can publish the signed request, so peers can see that it was submitted and is missing from the steward's history. Phloem has no clock to set a deadline, so this shows delay without proving refusal, and it is not a forfeit ground. Equivocation and unaccounted transfers are forfeit grounds (Section 6, σ_2), and any peer holding the conflicting records can take them to the adjudicators. If a steward goes silent, its vault stops closing cycles, and unvested rewards held in its balance stop paying. A fallback that lets registrants re-anchor their objects in another vault without losing their ancestry, and any guarantee of liveness or censorship resistance, are open (Section 10).

A vault contains the objects whose home is V and references to objects reachable by ancestor or descendant traversal up to depth τ. Peers may hold many overlapping vault views, and none needs the complete union. Imported objects inform local fertility and receive upstream mass from the paying vault's activations, but draw base reward only from their own home vault. Not specified in this version: splitting or merging vaults, and any cross-vault interaction beyond citing a parent in another vault.

3.3 Objects and provenance

A Phloem Object is $o = (id, H, M, Cap, p, sup)$, carried with an advisory timestamp t_{adv} outside the hashed content, with a 256-bit BLAKE3 *id*. Identity is the content hash alone. The provenance graph connects children to their listed parents with fork, mutation, or composition edges; supersedes edges form a separate version chain per creator. Local views must be acyclic across both edge kinds, and a cycle invalidates the registration that closes it. Cross-vault references are acyclic by construction, because a parent must already have an accepted registration record before a child can cite it.

The timestamp t_{adv} is the creator's unauthenticated claim, kept for display. It is outside the hashed bytes, so it never enters ids, spent-set keys, fertility, or acceptance.

3.4 Reconciliation

Combining two views of a vault is a pure function of records: take the union of records with valid signatures and inclusion proofs, discard the rest, and merge duplicate ids, which carry identical bytes. Bytes that do not hash to their claimed id are discarded. No id can carry two contents; a changed object is a new object with a supersedes edge. A vault has one steward history, so its spent set is never merged with or imported from another vault. Two conflicting histories signed by one steward are evidence of steward equivocation.

4. Fertility components

Branching. $B(o)$ sums over descendants $d \in D(o)$ the term $\alpha^{\text{depth}(d)} \cdot \text{nov}(d) \cdot \mathbf{1}[d \text{ active}]$, where $\text{depth}(d)$ is the shortest hop count from o to d. A descendant is active if it was registered, or gained a descendant of its own, within the last W_{act} activation cycles. The novelty factor $\text{nov}(d)$ is 0 for a near-copy, whose similarity to some parent exceeds θ_{sim}, and otherwise one minus its largest similarity to a parent (Appendix A.2). Here $\text{sim} \in [0, 1]$ is a similarity function over specifications, fixed by the parameter set. This version does not choose one: candidates such as token-set Jaccard similarity over the formal specification, or cosine similarity under an embedding model whose weights the parameter set commits to, are unevaluated (Section 10). Until a parameter set fixes sim, $\text{nov}(d) = 1$.

Past N_{thresh} active descendants, branching is damped. With $D_{\text{act}}(o)$ the set of active descendants and $\eta = 0.5$,

$$B_{\text{eff}}(o) = B(o) \cdot \Big(\frac{N_{\text{thresh}}}{\max(N_{\text{thresh}}, |D_{\text{act}}(o)|)} \Big)^{\eta}$$

B grows roughly linearly in $|D_{\text{act}}|$, so beyond the threshold B_{eff} grows like $\sqrt{N_{\text{thresh}}|D_{\text{act}}|}$. More descendants always count, at a falling rate, and an object with 5,000 active descendants still outscores one with 50.

New capabilities. If $U(d)$ is the set of capabilities first verified at d, meaning verified at d and at no ancestor of d, then o is credited $s(d, o)$ for each capability in $U(d)$ of each direct child d.

Dependency centrality. $Dep(o)$ counts descendants d with $U(d) \neq \emptyset$ that o dominates: o lies on every provenance path from d to a root, so removing o would cut d off from its foundations. The count n enters as $\log(1 + n)$ and is then max-normalized, so one dominated descendant no longer earns the top score. A vault's root dominates every descendant whose ancestry stays inside the vault, so roots score high on Dep by construction. That is intended, since everything in the vault builds on the root. A direct parent can receive both C and Dep for the same capability; a deeper ancestor receives only Dep.

Diversity. $Div(o)$ is the Shannon entropy of the object_type field (Appendix B) over the active descendants of o, and 0 if there are none.

Persistence. $P(o) = \sum_{h \leq h_{\text{now}}} \exp(-\lambda\,(h_{\text{now}} - h))\, u(o, h)$, where h indexes activation cycles of the home vault and $u(o, h)$ is the number of registrations accepted in cycle h that have o as an ancestor within depth τ. The decay λ is per activation cycle and not yet calibrated. Because P accumulates, an old object with steady use keeps a high score; Section 9 measures how concentrated rewards become as a result.

Multiple parents do not inflate rewards. Fertility components are scores, and emissions are governed by the per-hop split of Section 2.4, whose total is bounded independently of DAG shape.

5. Incentives and security

Named adversaries:

- (A1) vault manipulation with many identities and spurious objects;
- (A2) ancestry forgery, citing a parent that does not exist or was not used, including citing an unneeded parent with a small id to steer which vault becomes home;
- (A3) self-farming, registering a root and many dummy descendants (near-duplicate forks) to collect their base rewards and route upstream mass back to the root;
- (A4) poisoned conceptual ancestry;
- (A5) repeated settlement of one activation;
- (A6) capability spoofing, declaring capabilities an object does not have;
- (A7) a self-dealing steward that registers Sybil objects in its own vault and closes activation cycles with them;
- (A8) parent omission, registering a copy or near-copy of an existing object as a new root or with incomplete parents.

Spam cost. The fee c_{prop} scales with the vault's growth per activation cycle (Appendix A.5) and is paid to the treasury. Once a parameter set fixes sim, near-copies of a parent add little or nothing to B, and past N_{thresh} active descendants B_{eff} grows only as a square root (Section 4). Self-farming (A3) meets several partial limits. Each dummy descendant costs a fee paid to the treasury, and each new identity locks $\theta_{\min}$ of collateral in the vault, although a valid dummy fork breaks no rule and forfeits nothing. Once sim is fixed, a dummy above θ_{sim} adds nothing to its parent's B. Dummies of one object type add nothing to *Div*, and a dummy with no new capability adds nothing to C or *Dep*. Upstream mass is capped by the cascade: a dummy passes at most ρ of its base reward per hop. A dummy still earns its own base share, raised by the B and P that further dummies below it contribute, and that share grows with their number. We do not yet have a bound on the reward an adversary controlling k identities can extract from a vault this way (A1, A3). Deriving one is the main open proof obligation (Section 10). New vaults are a second Sybil surface: registering a root costs one fee and opens a vault with its own emissions schedule. The schedule is therefore a ceiling: a vault receives SAP only if the treasury funds it, and the rule that decides which vaults are funded is part of the open allocation rule.

Ancestry forgery (A2). A cited parent must already have a registration record, so citing a nonexistent parent forfeits collateral (Section 6, σ_1), and citation rings cannot form because a parent is always registered before the child that cites it. Citing a real but unneeded parent is harder to prove. Its main effect is to route cascade mass to the cited object, and a child can route at most a fraction ρ of its own base reward this way, a quarter at the default.

Self-dealing steward (A7). The steward decides which registrations to record, and 100 recorded registrations close a cycle. Each registration record must reference its fee transfer to the treasury, so the steward cannot count unpaid registrations. Closing a cycle entirely with its own objects therefore costs at least $100\,c_{\text{prop}}$ and returns at most $\mathcal{E} < R_{\text{activation}}/(1-\rho)$. At $c_{\text{prop}} = c_{\text{base}} = 10$ SAP this is unprofitable whenever $R_{\text{activation}} \leq 750$ SAP. Under that condition the vault as a whole collects at least as much in fees as it pays out, so honest participants are paid only from what other registrants pay in. A vault that pays more per cycle, as the toy example does, relies on two weaker limits: Sybil objects earn only their fertility share, which the open bound has to cap, and the treasury's allocation to a vault caps what self-dealing can extract from it in total (Section 6).

Repeated settlement (A5) is excluded by rule and detectable: settlement is valid only in the home vault, the spent-set check of Section 2.5 rejects a second settlement of the same activation, and a steward that pays twice signs evidence against itself. Integrity checks on accept also cover the content hash, known parents or proofs, acyclicity, same-creator supersession, the creator's DSM device identity, and signature validity. There is no consensus protocol; each peer checks the same deterministic predicates.

Capability spoofing (A6) is enforceable only in part. A declared capability whose predicate, re-executed on the committed artifact under the inputs and environment the parameter set specifies, returns 0 is provably false, and its creator forfeits collateral (Section 6, σ_3). A claim that passes the predicate without having the capability, for example by overfitting the predicate's test

inputs, cannot be shown false, because there is no oracle of true capability. Phloem does not penalize such claims; it limits what they pay. C goes to direct parents only, and *Dep* and the other components depend on descendants that other peers choose to build, so a capability that does not hold up attracts few productive descendants and little fertility. The remaining gap is listed in Section 10.

Parent omission (A8). Content hashing cannot detect a copy, because the id covers the creator identity, and θ_{sim} compares a child only with its declared parents. Partial mitigations exist. Any peer can compare a new object's specification with objects in other vaults using the same similarity function and publish a flag. Within one vault, registration order is signed, so the earlier of two near-identical objects is provable. Across vaults there is no shared clock, so priority has to rest on an earlier commitment to the original's id, published in a vault record or an external timestamping service before the copy appeared, or on the copier's own history showing that it imported the original. A challenge rule that turns such evidence into a reassignment of credit is not specified (Section 10). Until it is, attribution in Phloem depends, as in citation, on registrants declaring their parents and on peers calling out omissions.

6. Token role (SAP)

SAP is Phloem's unit of attribution credit. It is a DSM token whose committed policy fixes its rules: ticker SAP, 6 decimals (divisibility 10^{-6}), and a total supply of 10^{10} SAP. In the current DSM beta, a token created by a device releases its whole supply to the creating account at creation, and no unit can be minted later [7]. The creating account therefore holds the whole supply at creation. This is a DSM constraint, not an allocation. This version specifies no allocation to the author or to any team. A treasury account receives 10^8 SAP to fund vault stewards and collects all registration fees. How those 10^8 SAP are divided among vaults is not specified. The remaining 9.9×10^9 SAP stays in the creating account, unallocated, until an allocation rule is specified (Section 10). Until then, the treasury and the unallocated supply sit under the creator's keys, a trust assumption that the allocation rule has to remove. The same rule has to say how first registrants obtain the SAP they need for fees and collateral, since this version distributes SAP only as rewards. SAP will be identified by its policy commitment, published when the token is created, because DSM does not reserve tickers.

The emissions schedule is Phloem policy, applied by the treasury and the stewards per vault:

$$R_{\text{activation}}(h) = R_0 \cdot \exp\Big(-\frac{\ln 2 \cdot h}{T_{1/2}}\Big) \cdot \min\Big(1, \frac{F_{\text{total}}(h)}{F_{\text{target}}}\Big)$$

Here h is the vault's activation cycle index, R_0 the per-cycle budget at $h = 0$, $T_{1/2}$ the half-life in cycles, $F_{\text{total}}(h) = \sum_{o:\,\text{home}(o)=V} F(o)$ the vault's total fertility at cycle h, and F_{target} the total at which a vault receives its full budget. The last factor keeps a nearly empty vault from drawing a full budget; it grows with object count, so it does nothing against Sybil registration. R_0 and F_{target} are set per parameter set and have no proposed values, since they depend on the supply allocation. $T_{1/2} = 2.1 \times 10^6$ cycles is a placeholder. Each activation pays at most

$R_{\text{activation}}/(1-\rho)$ from the steward's SAP balance (Section 2.4). DSM enforces the supply cap and the conservation of every transfer; peers check the schedule against the published settlement records. All amounts are integers in base units of 10^{-6} SAP, and exp and ln are evaluated by a fixed-point rule published with the parameters.

Reward credits vest linearly over $\nu = 24$ activation cycles. DSM has no time lock, so the steward pays each vested tranche as a separate transfer when it vests, and the unvested remainder stays in the steward's balance. Besides credit, SAP pays registration fees, in addition to the DSM storage credits every writing account pays, and backs integrity collateral of at least $\theta_{\min} = 10000$ SAP, posted once per vault by the steward and by each registrant.

Integrity collateral uses DSM escrow vaults. An escrow vault holds one amount of one token and releases the whole amount once, to the recipient of one precommitted branch, when every signer of that branch has signed [7]. A depositor locks $\theta_{\min}$ in two escrow vaults, one of $\sigma_1\theta_{\min}$ and one of $(1-\sigma_1)\,\theta_{\min}$. Each has a refund branch, signed jointly by the depositor and the adjudicators, and a forfeit branch to the treasury, signed by the adjudicators alone. The adjudicators are one to four keys named by the vault's parameter set; the depositor does not choose them. They sign a forfeit only on signed evidence that anyone can check:

- ancestry violation ($\sigma_1 = 0.01$): a registration signed by the depositor whose bytes do not hash to its id, or that cites a parent with no record; releases the smaller vault;
- steward misconduct ($\sigma_2 = 1.0$): two conflicting records signed by the depositor as steward, or a SAP transfer from the steward account that no settlement record in its signed history accounts for; releases both vaults;
- failed capability claim ($\sigma_3 = 1.0$): a registration signed by the depositor that declares a capability whose predicate, re-executed on the committed artifact as the parameter set specifies, returns 0; releases both vaults.

Predicates that cannot be re-executed deterministically are outside σ_3 (Section 5). DSM enforces only that each escrow vault pays one branch, once. The adjudicators are trusted for the verdict, and a deposit with no verdict stays locked. The treasury funds a vault in tranches no larger than its steward's collateral, so a steward caught diverting a tranche forfeits at least what it took.

Parameter sets. A vault's parameters (component weights, split rule, sim, capability names and predicates, adjudicator keys, half-life, and base fee) are fixed in its root registration, which selects one published parameter set. A new parameter set applies only to vaults created under it; existing vaults keep theirs. DSM has no governance mechanism, and a committed token policy cannot change, so how parameter sets are published and chosen is a Phloem process outside DSM, not yet specified (Section 10).

SAP has no function outside Phloem's accounting: it records attribution credit, pays protocol fees, and backs integrity collateral. This specification defines no sale or distribution of SAP beyond those uses.

7. Implementation outline

No code exists. The intended peer is written in Rust and keeps vault state in an application-level sparse Merkle tree, separate from DSM's per-device tree. Planned modules cover vault state, provenance queries (descendants and dominators), score recomputation, and settlement with the spent set.

Operation	**Time**	**Space**
RegisterObject	$O(\|p\|)$	$O(1)$
GetDescendants (depth τ)	$O(\|D(o)\|)$	$O(\|D(o)\|)$
ComputeFertility	$O(\|V\|\,\bar{d}_\tau + \|E\|\log\|V\|)$ plus similarity	$O(\|V\| + \|E\|)$
RewardPropagation (τ hops)	$O(\tau \cdot (\|V\| + \|E\|))$	$O(\|V\|)$
Spent-set check	$O(\log n_{\text{spent}})$	$O(\log n_{\text{spent}})$
MerkleProof	$O(\log\|V\|)$	$O(\log\|V\|)$

Here $|E|$ is the number of provenance edges among the vault's objects, $\bar{d}_\tau$ the mean number of descendants within depth τ, and n_{spent} the size of the spent set. Dominators for *Dep* come from the Lengauer-Tarjan algorithm [9] on the parent-to-child graph, started from a virtual super-root joined to every root. Similarity cost depends on the function a parameter set fixes; comparing each child only with its parents takes $O(|E|)$ comparisons. Performance target: fertility over about 100k objects and 500k edges in under two seconds (target; not yet measured).

Phloem needs one guarantee from its value layer: a paid credit cannot be spent twice. DSM gives it to an application with its own DSM account [8]. A global blockchain [10, 11, 12] would also work but adds validators, block ordering, and gas that single-writer vaults do not need.

8. Related work

Dependency-graph funding. Deep Funding [13, 14], an Ethereum public-goods experiment initiated by V. Buterin, weights each edge of a software dependency graph by the fraction of a project's credit that belongs to the dependency, and lets each node keep part of its funding and pass the rest down. Drips [15] lets a project route fixed percentages of its income to the projects it depends on, recursively. Phloem's per-hop split is closest to these two. The difference is how the split is set: Phloem fixes it by rule (ρ and s), Deep Funding sets edge weights through competing models and markets checked by a jury, and Drips leaves them to each maintainer. Phloem also scores architecture objects by the descendants they produce, where both systems fund software packages by judged importance or declared dependence.

Graph ranking. tea [16] ranks open-source packages with teaRank, a PageRank-style score over package dependency graphs, and pays them in a token with staking and slashing. SourceCred [17] runs PageRank over a graph of contributions and contributors. Phloem's fertility is a weighted mix of descendant-structure terms and is closer to bibliometric measures of forward citation and disruption, such as the CD index [18] and its use to show that papers and patents have become less disruptive [19].

Retroactive and plural funding. Optimism's retroactive public goods funding [20] pays projects after their impact is visible, as judged by a panel. Hypercerts [21] record impact claims that funders can buy or reward later. Quadratic funding [22] matches contributions in proportion to the breadth of support. All three rely on human judgment or contributions to decide value. Phloem replaces that judgment with a fixed rule over recorded provenance, which is cheaper to run and easier to game; Section 5 is about the second part.

Marginal-contribution attribution. Data Shapley [23] values training data by its average marginal contribution to model performance. A Shapley-style rule would replace the fixed ρ with marginal contributions, but it needs a performance oracle and repeated retraining, which peers cannot run deterministically over a vault. Phloem accepts a cruder fixed split in exchange for a rule every peer can recompute.

AI incentive markets and storage. Bittensor [24] has peers rank one another's models by learned informational value and pays for current model output; Phloem pays for architectural provenance. Neural architecture search [3, 4] optimizes accuracy under centralized evaluation. Content-addressed storage [25] and permanent storage [26] provide persistence with no reward for downstream use. Huberman, Romero, and Wu [27] find that attention drives contribution, which motivates paying for downstream uptake. Ramsay's Deterministic Provenance proposal [28] treats origin and derivation as properties of state on DSM; Phloem adds a credit rule for what is derived.

9. Evaluation plan

Proposed evaluations, none run yet: Spearman correlation between graph centrality and cumulative reward; Gini coefficient and tail mass of rewards under fertility-weighted and uniform scoring, and how that concentration grows over cycles as persistence accumulates; branching and depth distributions against random forks; equal against fertility-weighted split on synthetic DAGs; and adversarial simulations of self-forking, similarity spam, capability spoofing, a self-dealing steward, parent omission, and repeated settlement, sweeping c_{prop} and $R_{\text{activation}}$.

10. Open problems and proof obligations

Specified in this version: the multi-parent split and its emissions bound on arbitrary DAGs (Section 2.4), home-vault settlement with spent sets (Section 2.5), the steward model (Section 3.2), content-hash identity with supersedes edges (Sections 2.1, 3.3), clockless acceptance (Section 3.1), and one credit rule for C and Dep (Sections 2.3, 4). Still open:

- Bound the reward an adversary controlling k identities can extract from a vault, as a function of c_{prop}, θ_{sim}, and N_{thresh}, covering Sybil descendants (A1, A3) and a self-dealing steward (A7).
- Specify a challenge rule for parent omission (A8): what proves similarity to an earlier object, how priority is shown across vaults without a shared clock, and how credit is then reassigned.

- Specify an allocation rule for the SAP supply: how the treasury divides funds among vaults and which vaults are funded at all, how first registrants obtain SAP, and what happens to the 9.9×10^9 unallocated SAP. Until then the creating account is a single point of trust.
- Choose and evaluate the similarity function sim, and define Val and S or remove them.
- Calibrate the fertility weights, ρ, λ, R_0, F_{target}, and $T_{1/2}$ together, given that ancestors gain from descendants through both F and the cascade, so that a scoring failure pays less (a component that cannot be computed scores zero).
- Bound capability spoofing beyond the provable case of Section 6, including predicates that can be passed by overfitting their test inputs.
- Specify how parameter sets exclude capability classes with dual-use or safety implications; the protocol should never credit disclosure of a hazardous capability.
- Specify steward selection, exit, and replacement, how registrants re-anchor in another vault when a steward refuses or goes silent, what liveness and censorship resistance a vault can guarantee, and what peers can prove about a steward from its signed records.
- Specify how parameter sets are published and adopted, and who holds their adjudicator keys, given that DSM has no governance mechanism and SAP's policy cannot change; or replace adjudication with a program DSM can evaluate once that is possible.
- Evaluate moving registrations onto DSM records once DSM ships the durable signed records and verifiable collections its beta roadmap targets for the next release.

11. Conclusion

Phloem pays architecture primitives for the downstream work recorded in their provenance. Whether it holds up depends on Section 10, chiefly the Sybil bound and the supply allocation. The next step is to simulate the adversarial tests of Section 9 before building anything.

Nothing in the reward rule is specific to AI. The split, its emissions bound, and home-vault settlement apply to any work whose lineage can be recorded, such as open-source software, datasets, or scientific methods. This paper specifies the AI architecture case, and the capability predicates are the main part another field would have to define for itself.

12. Contact and collaboration

Counterexamples to anything in Section 10 are the most useful thing you can send. Contact hello@pinealab.com, or @pepedzekichen on X, Telegram, and Discord.

Disclaimer. This document describes a protocol design. It is not an offer or solicitation of any token or financial instrument, and SAP has not been issued.

Appendix A. Equations and parameters

A.0 Notation

Symbol	Meaning
o, id, M, H, Cap	object, its content hash, specification, creator DSM identity, declared capabilities
$p(o)$, $sup(o)$, $A(o)$, $D(o)$	parents, superseded version, all ancestors, descendants within depth τ
V, $\lvert V\rvert$, $\lvert E\rvert$, home(o)	vault, number of objects whose home is V, provenance edges in V, paying vault of o
h, $(V, \ell, prev)$	activation cycle index; vault, position, predecessor hash of a record
F, B_{eff}, C, Div, P, Val, S, Dep	fertility and its components (Section 4)
nov, sim, $u(o,h)$, $U(d)$	novelty factor, similarity, registrations under o in cycle h, capabilities first verified at d
$s(c,a)$, ρ, τ, m_k, μ_k, $\mathcal{E}$	split weight, upstream ratio, max depth, mass at hop k, total mass at hop k, emission per activation
$R_{\text{activation}}$, R_0, F_{total}, F_{target}, $T_{1/2}$	budget per activation, initial budget, vault total fertility, full-budget level, half-life
c_{prop}, $\theta_{\min}$, $\sigma_1, \sigma_2, \sigma_3$, ν	registration fee, collateral, forfeiture rates, vesting cycles
$K(act)$, n_{spent}	spent-set key of an activation, spent-set size

A.1 Identity and home vault

$$id = \text{BLAKE3-256}\Big(\texttt{"phloem/object/v1\0"} \parallel \text{CanonicalSerialize}(M, H, Cap, p, sup)\Big)$$

p sorted; sup = null or a prior id

$$\text{home}(o) = \begin{cases} V(o) & \text{if } p(o) \text{ is empty} \\ \text{home}(\min p(o)) & \text{otherwise} \end{cases}$$

$sup(o') = id(o)$ is valid only if $H(o') = H(o)$

Phloem hashes are domain-separated as BLAKE3-256($tag \parallel$ `0x00` $\parallel data$) with tags under `"phloem/"`, following DSM's convention; the `"DSM/"` namespace is reserved to DSM. Where a Phloem commitment must be bound into a DSM transfer, it enters as DSM's external commitment $Y = \text{BLAKE3-256}(\texttt{"DSM/external/v1"} \parallel X)$ [6]. The advisory timestamp t_{adv} is not hashed.

A.2 Fertility

$$F(o) = \sum_{i=1}^{7} w_i F_i(o), \qquad (F_1, \ldots, F_7) = (B_{\text{eff}}, C, Div, P, Val, S, Dep)$$

each F_i max-normalized to $[0,1]$ over the objects whose home is the vault

$$B(o) = \sum_{d \in D(o)} \alpha^{\mathrm{depth}(d)} \cdot \mathrm{nov}(d) \cdot \mathbf{1}[d \text{ active}]$$

$$\mathrm{nov}(d) = \begin{cases} 0 & \text{if } \max_{a \in p(d)} \mathrm{sim}(d,a) > \theta_{\mathrm{sim}} \\ 1 - \max_{a \in p(d)} \mathrm{sim}(d,a) & \text{otherwise} \end{cases}$$

$$B_{\mathrm{eff}}(o) = B(o) \cdot \Big(\frac{N_{\mathrm{thresh}}}{\max(N_{\mathrm{thresh}}, |D_{\mathrm{act}}(o)|)} \Big)^{\eta}$$

$$U(d) = \{\, c : \mathrm{ver}_c(d) = 1 \text{ and } \mathrm{ver}_c(a) = 0 \text{ for all } a \in A(d) \,\}, \qquad C(o) = \sum_{d:\, o \in p(d)} s(d,o) \cdot |U(d)|$$

$$\mathit{Dom}(d) = \{\, a \in A(d) : a \text{ lies on every provenance path from } d \text{ to a root} \,\}$$

$$\mathit{Dep}(o) = \log\Big(1 + |\{\, d \in D(o) : U(d) \neq \emptyset \text{ and } o \in \mathit{Dom}(d) \,\}|\Big)$$

$$\mathit{Div}(o) = -\sum_{t} \pi_t(o) \log \pi_t(o), \qquad \pi_t(o) = \text{share of active descendants of } o \text{ with object_type } t$$

$$P(o) = \sum_{h \le h_{\mathrm{now}}} \exp\Big(-\lambda\,(h_{\mathrm{now}} - h) \Big) \cdot u(o,h)$$

Val and S are not defined in this version and have weight 0.

A.3 Rewards and emissions bound

$$R_{\mathrm{base}}(o) = R_{\mathrm{activation}} \cdot \frac{F(o)}{\sum_{o':\, \mathrm{home}(o') = V} F(o')}$$

$$s(c,a) = \begin{cases} 1/|p(c)| & \text{default} \\ \dfrac{F(a)}{\sum_{a' \in p(c)} F(a')} & \text{fertility-weighted option} \\ & (F \text{ from previous cycle; equal if parents} \\ & \text{span vaults, } F \text{ unavailable, or sum zero}) \end{cases}$$

$$\begin{aligned} m_0(x) &= R_{\mathrm{base}}(x) \\ m_k(a) &= \rho \sum_{c:\, a \in p(c)} s(c,a)\, m_{k-1}(c), \qquad k = 1, \ldots, \tau \\ R_{\mathrm{total}}(a) &= R_{\mathrm{base}}(a) + \textstyle\sum_{k=1}^{\tau} m_k(a) \end{aligned}$$

$$\begin{aligned} \mu_k &= \textstyle\sum_x m_k(x) \le \rho\, \mu_{k-1}, \qquad \mu_0 = R_{\mathrm{activation}} \\ \mathcal{E} &= \sum_{k=0}^{\tau} \mu_k \le R_{\mathrm{activation}} \cdot \frac{1 - \rho^{\tau+1}}{1 - \rho} < \frac{R_{\mathrm{activation}}}{1 - \rho} \end{aligned}$$

$$\rho = 0.25,\ \tau = 5: \quad \mathcal{E} \le \tfrac{1365}{1024}\, R_{\mathrm{activation}} \approx 1.33301\, R_{\mathrm{activation}} < \tfrac{4}{3}\, R_{\mathrm{activation}}$$

A.4 Settlement

$$K(act) = \text{BLAKE3-256}\big(\texttt{"phloem/spent/v1\textbackslash 0"} \,\|\, \text{record_id}(act)\big)$$

valid Settle(act) in $V \iff$ home(act) $= V$ and record(act) $\in$ history(V)
and $K(act) \notin$ SpentSet(V) before
and amounts = recomputation of A.3

after: SpentSet(V) := SpentSet(V) $\cup \{K(act)\}$
(checked by peers; each payout is a DSM transfer of SAP from the steward)

A.5 Registration fee and self-dealing condition

$$c_{\text{prop}}(h) = c_{\text{base}} \cdot \Big(1 + \gamma \cdot \frac{|V_h| - |V_{h-1}|}{|V_{h-1}|}\Big), \qquad |V_h| = \text{objects in } V \text{ at the close of cycle } h$$

steward closing a cycle with only its own objects: cost $\geq 100\, c_{\text{prop}}$, gain $\leq \mathcal{E} < \dfrac{R_{\text{activation}}}{1-\rho}$

A.6 Emissions schedule

$$R_{\text{activation}}(h) = R_0 \cdot \exp\Big(-\frac{\ln 2 \cdot h}{T_{1/2}}\Big) \cdot \min\Big(1, \frac{F_{\text{total}}(h)}{F_{\text{target}}}\Big), \qquad F_{\text{total}}(h) = \sum_{o:\, \text{home}(o)=V} F(o)$$

A.7 Parameter defaults

Parameter	Default	Changeable in a new parameter set
α (branching decay)	0.85	yes
λ (persistence decay)	per activation cycle; not yet calibrated	yes
ρ (upstream ratio)	0.25	yes
τ (max depth)	5	yes
split rule $s(c, a)$	equal	yes
c_{base} (base fee)	10 SAP	yes
γ (fee responsiveness)	1.0	yes
θ_{sim} (near-copy threshold)	0.85	yes
η (branching damping)	0.5	yes
N_{thresh}	50	yes
W_{act} (activity window)	24 cycles	yes
R_0, F_{target}	none proposed	yes
activation length	100 registrations	no
$T_{1/2}$ (half-life)	2.1×10^6 cycles (placeholder)	no
$\theta_{\min}$ (integrity collateral)	10000 SAP	yes
σ_1 ancestry violation	0.01	yes
σ_2 steward misconduct	1.0	no
σ_3 failed capability claim	1.0	no
ν reward vesting	24 cycles	yes

A parameter marked yes can differ between parameter sets (Section 6). It never changes inside an existing vault.

Appendix B. PhloemObject interchange schema (off-core)

This JSON form is for tooling and display, and DSM never parses it. The bytes hashed into id are the canonical binary encoding of the same fields, with ids as raw 32-byte values and without the timestamp. DSM identities are shown in Base32 Crockford, as DSM does at user-facing boundaries.

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "title": "PhloemObject",
  "$comment": "id = BLAKE3-256(\"phloem/object/v1\\0\" || canonical bytes), timestamp excluded",
  "description": "Immutable; an edit is a new object.",
  "type": "object",
  "required": ["spec_version", "creator", "specification", "capabilities",
               "parents", "supersedes"],
  "properties": {
    "spec_version": {"type": "string", "pattern": "^1\\.[0-9]+$"},
    "creator": {
```

```
      "type": "object", "required": ["genesis", "device_id"],
      "properties": {
        "genesis":   {"type": "string", "pattern": "^[0-9A-HJKMNP-TV-Z]{52}$"},
        "device_id": {"type": "string", "pattern": "^[0-9A-HJKMNP-TV-Z]{52}$"}
      }
    },
    "specification": {
      "type": "object",
      "required": ["object_type", "name", "description", "formal_spec", "reference_impl"],
      "properties": {
        "object_type": {
          "type": "string",
          "enum": ["neural_topology", "optimizer", "reasoning_framework",
                   "memory_architecture", "simulation_env", "capability_interface"]
        },
        "name": {"type": "string", "maxLength": 128},
        "description": {"type": "string", "maxLength": 4096},
        "formal_spec": {
          "type": "object",
          "required": ["format", "content_uri"],
          "properties": {
            "format": {"enum": ["latex", "pseudocode", "json", "yaml"]},
            "content_uri": {"type": "string", "pattern": "^ipfs://[a-zA-Z0-9]+$"}
          }
        },
        "reference_impl": {
          "type": "object",
          "required": ["language", "repository_uri", "commit_hash"],
          "properties": {
            "language": {"type": "string"},
            "repository_uri": {"type": "string", "format": "uri"},
            "commit_hash": {"type": "string", "pattern": "^[a-f0-9]{40}$"}
          }
        }
      }
    },
    "capabilities": {
      "description": "Names defined in the vault's parameter set; others score nothing.",
      "type": "array", "items": {"type": "string", "maxLength": 64}
    },
    "parents": {
      "type": "array",
      "items": {"type": "string", "pattern": "^[a-f0-9]{64}$"},
      "maxItems": 16,
      "uniqueItems": true
    },
    "supersedes": {
      "description": "Prior version by the same creator, or null. No reward.",
      "type": ["string", "null"], "pattern": "^[a-f0-9]{64}$"
    },
    "timestamp": {
      "description": "Display only. Not hashed and never read by validity checks.",
      "type": "integer", "minimum": 0
    }
  }
}
```

Appendix C. References

[1] B. Ramsay, Irrefutable Labs. Deterministic State Machines as Guarded Linear Constraint Systems: Double-spend prevention as a state property. Cryptology ePrint Archive, Paper 2025/592, 2025, revised July 11, 2026. https://eprint.iacr.org/2025/592

[2] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin. Attention is all you need. In Advances in Neural Information Processing Systems 30, pages 5998–6008, 2017.

[3] B. Zoph and Q. V. Le. Neural architecture search with reinforcement learning. In International Conference on Learning Representations, 2017.

[4] E. Real, A. Aggarwal, Y. Huang, and Q. V. Le. Regularized evolution for image classifier architecture search. In Proceedings of the AAAI Conference on Artificial Intelligence, 33(1):4780–4789, 2019.

[5] National Institute of Standards and Technology. Stateless Hash-Based Digital Signature Standard. FIPS 205, August 2024. https://doi.org/10.6028/NIST.FIPS.205

[6] DSM High Level Explainer, amended Markdown edition (Amendments A11, A21, A23). Deterministic State Machine repository, November 2026. https://github.com/deterministicstatemachine/dsm/blob/main/specs/DSM_High_Level_Explainer.md

[7] Sovereign Finance (SoFi) Settlement Specification, Part IX (token policy) and Section 19.9 (escrow vaults). Deterministic State Machine repository, November 2026. https://github.com/deterministicstatemachine/dsm/blob/main/specs/SoFi_Settlement_Specification.md

[8] BUILDING_APPS.md. Deterministic State Machine repository, November 2026. https://github.com/deterministicstatemachine/dsm/blob/main/BUILDING_APPS.md

[9] T. Lengauer and R. E. Tarjan. A fast algorithm for finding dominators in a flowgraph. ACM Transactions on Programming Languages and Systems, 1(1):121–141, 1979.

[10] S. Nakamoto. Bitcoin: A peer-to-peer electronic cash system. Cryptography Mailing List, 2008.

[11] V. Buterin. A next-generation smart contract and decentralized application platform. Ethereum Whitepaper, 2014.

[12] J. Kwon and E. Buchman. Cosmos: A network of distributed ledgers. Cosmos Whitepaper, 2016.

[13] Deep Funding. Project site, https://www.deepfunding.org (accessed November 2026).

[14] E. Yeung, C. Lesaege, and D. Mehta. Deep Funding: A prediction market for open source dependencies. Ethereum Research forum, September 2025. https://ethresear.ch/t/deep-funding-a-prediction-market-for-open-source-dependencies/23101

[15] Drips documentation. https://docs.drips.network (accessed November 2026).

[16] M. Howell and T. Lewis. Tea Network white paper: From dApp to OP Stack Layer 2, version 1.9. https://docs.tea.xyz/tea-white-paper/white-paper (accessed November 2026).

[17] SourceCred. An introduction to calculating cred and grain. SourceCred documentation, https://github.com/sourcecred/docs/blob/main/intro_calculating_cred_grain.md (accessed November 2026).

[18] R. J. Funk and J. Owen-Smith. A dynamic network measure of technological change. Management Science, 63(3):791–817, 2017. https://doi.org/10.1287/mnsc.2015.2366

[19] M. Park, E. Leahey, and R. J. Funk. Papers and patents are becoming less disruptive over time. Nature, 613:138–144, 2023. https://doi.org/10.1038/s41586-022-05543-x

[20] Optimism, with V. Buterin. Retroactive public goods funding. Optimism blog, July 20, 2021. https://medium.com/ethereum-optimism/retroactive-public-goods-funding-33c9b7d00f0c

[21] H. Brammer. Hypercerts: A new primitive for public goods funding. Protocol Labs blog, August 24, 2022. https://protocol.ai/blog/hypercert-new-primitive/

[22] V. Buterin, Z. Hitzig, and E. G. Weyl. A flexible design for funding public goods. Management Science, 65(11):5171–5187, 2019. https://doi.org/10.1287/mnsc.2019.3337

[23] A. Ghorbani and J. Zou. Data Shapley: Equitable valuation of data for machine learning. In Proceedings of the 36th International Conference on Machine Learning, PMLR 97:2242–2251, 2019.

[24] Bittensor: A peer-to-peer intelligence market. Bittensor whitepaper, https://www.bittensor.com/whitepaper (accessed November 2026). An earlier version by Y. Rao, J. Steeves, A. Shaabana, D. Attevelt, and M. McAteer, arXiv:2003.03917, was withdrawn in 2021.

[25] J. Benet. IPFS: Content addressed, versioned, P2P file system. arXiv:1407.3561, 2014.

[26] S. Williams, V. Diordiiev, L. Berman, I. Raybould, and I. Uemlianin. Arweave: A protocol for economically sustainable information permanence. Arweave Yellow Paper, Draft 1, 2019.

[27] B. A. Huberman, D. M. Romero, and F. Wu. Crowdsourcing, attention and productivity. Journal of Information Science, 35(6):758–765, 2009.

[28] B. Ramsay. Deterministic Provenance: Origin Is a Property of State, Not Appearance. An open proposal, September 2026. https://www.deterministicstatemachine.org/Deterministic_Provenance.pdf